DSA STUDY BLUEPRINT SERIES

14. Book Allocation Problem

Minimizing the Maximum Pages Allocated to Students

Section 02 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Allocate contiguous books such that max pages allocated to a student is minimized.
- **Search Space:** Bounds lie between `max(pages)` and `sum(pages)`.
- Validate candidate mid pages using a helper validator.

Memory & Execution Blueprint

Search Space Bounds for Array [12, 34, 67, 90] (Students = 2):

Low = 90 (Max element) ← Binary Search Space → **High = 203** (Sum of elements)

C++ Implementation Reference

Standard code layout and syntax

```
bool isValid(vector& arr, int n, int students, int maxPages) {
    int studentCount = 1, pagesSum = 0;
    for (int p : arr) {
        if (p > maxPages) return false;
        if (pagesSum + p > maxPages) {
            studentCount++;
            pagesSum = p;
            if (studentCount > students) return false;
        } else pagesSum += p;
    }
    return true;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(N log(Sum-Max))
Space Complexity	O(1)

Key Practices & Gotchas

- **Important:** Binary search on answers is the primary strategy for optimization search spaces.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run Binary Search on Answers range checks (e.g. allocating book pages):

Iteration	Check Bounds [Low, High]	Mid Candidate Value	Feasibility Check: `isValid(mid)`	Search Window Adjustment
1	[90, 203]	mid = 146	True (Students needed ≤ K)	Shrink search: high = mid - 1 = 145
2	[90, 145]	mid = 117	False (Students needed > K)	Expand search: low = mid + 1 = 118

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Minimizing the Maximum:** Used when the answer search space is monotonic (if X is feasible, all values > X are also feasible). Search binary-wise for bounds limits.
- **Feasibility Helpers:** Write custom O(N) linear scan checks to evaluate candidate value viability.
- **Related LeetCode Problems:** #410 Split Array Largest Sum, #1011 Capacity To Ship Packages Within D Days, #875 Koko Eating Bananas.